A Demonstration of mLoRA: A Parallelism-Efficient LLM Fine-Tuning System

Zelong Huang*
zelonghuang05@gmail.com
Sichuan University
Chengdu, China

Zhengmao Ye*
yezhengmaolove@gmail.com
Sichuan University
Chengdu, China

Salma Filali
salmaf999o@gmail.com
Sichuan University
Chengdu, China

Qingsong Cai
qingsong1010@gmail.com
Sichuan University
Chengdu, China

Lei Duan
Jie Zuo
{leiduan,zuojie}@scu.edu.cn
Sichuan University
Chengdu, China

Jiale Lao
jiale@cs.cornell.edu
Cornell University
Ithaca, USA

Hui Lu
hui.lu@uta.edu
The University of Texas at Arlington
Arlington, USA

Mingjie Tang
tangrock@gmail.com
Sichuan University
Chengdu, China

ABSTRACT

Under the *pretrain-then-finetune* paradigm, large language models (LLMs) based on Transformer architectures and Low-Rank Adaptation (LoRA) techniques achieve excellent performance on a wide range of tasks at relatively low costs. This has led developers to fine-tune LLMs on service platforms for domain-specific downstream tasks. Fine-tuning many LoRA adapters on large models often requires model parallelism. However, existing parallel fine-tuning methods suffer from high communication overhead and inefficient GPU utilization, especially when distributing many LoRA tasks across multiple GPUs and machines.

This demonstration presents mLoRA, a system for parallel and efficient fine-tuning of LLMs. mLoRA effectively pipelines independent LoRA adapters across GPUs and machines, improving hardware utilization. It also introduces a LoRA-optimized operator that significantly enhances the efficiency of the fine-tuning process. This demonstration shows: (1) how mLoRA uses two core components to improve GPU utilization and task throughput in distributed settings; (2) how users can fine-tune many LoRA tasks in parallel to achieve higher GPU utilization and reduce task completion time under fixed resource constraints; and (3) how mLoRA supports database-related tasks, such as natural language to SQL (Text2SQL) and data preprocessing using LLMs (LLM4DP). The system implementation and demonstration are available at Code, along with a Video.

PVLDB Reference Format:
Zelong Huang, Zhengmao Ye, Salma Filali, Qingsong Cai, Lei Duan, Jie Zuo, Jiale Lao, Hui Lu, and Mingjie Tang. A Demonstration of mLoRA: A Parallelism-Efficient LLM Fine-Tuning System. PVLDB, 14(1): XXX-XXX, 2025.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code and video have been made available at https://github.com/TUDB-Labs/mLoRA and https://youtu.be/m2qMYlFmmfo.

1 INTRODUCTION

Pre-trained large language models (LLMs) based on Transformer architectures have demonstrated strong general performance. Fine-tuning these models with domain-specific data further improves their performance on specialized tasks, such as translating natural language questions into SQL queries for relational databases [3], converting heterogeneous data lakes into structured and queryable tables [1], and implementing open-source column type annotation methods using LLMs [2]. These applications commonly adopt a fine-tuning technique named Low-Rank Adaptation (LoRA), a state-of-the-art parameter-efficient fine-tuning (PEFT) technique. LoRA achieves efficient fine-tuning by freezing the pre-trained model and updating only low-rank additive matrices, significantly reducing the number of trainable parameters.

However, fine-tuning a high-performance LoRA adapter presents several challenges. The performance of a LoRA adapter is highly sensitive to its parameter settings (e.g. the rank and the scaling factor alpha of LoRA). Even small changes can result in significant performance differences. In addition, these hyperparameters span a large and complex search space. Training many adapters on a large base model often requires multiple GPUs, and distributing the base model and LoRA adapters among GPUs involves model parallelization. Existing model parallelization techniques (tensor parallelism and pipeline parallelism) suffer from high communication overhead due to intra-GPU, inter-GPU, or inter-machine synchronization, or from low GPU utilization due to pipeline bubbles. Moreover, the small size of LoRA adapters worsens this problem; parallel training of many small adapters leads to frequent GPU kernel launches, which can increase the total training time by up to 10%.

mLoRA[1] presents a novel and highly efficient framework for training adapters, significantly reducing fine-tuning time. To address the issue of pipeline bubbles, we use non-dependent adapters to fill bubbles in the pipeline to achieve zero bubbles. When the number of LoRA tasks is insufficient to fully support a zero-bubble pipeline, we employ a method similar to GPipe to eliminate bubbles efficiently, which divides the mini-batch into multiple (e.g.,

*The authors contributed equally to this paper.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

[1]Our research paper (mLoRA) has been accepted by VLDB 2025.

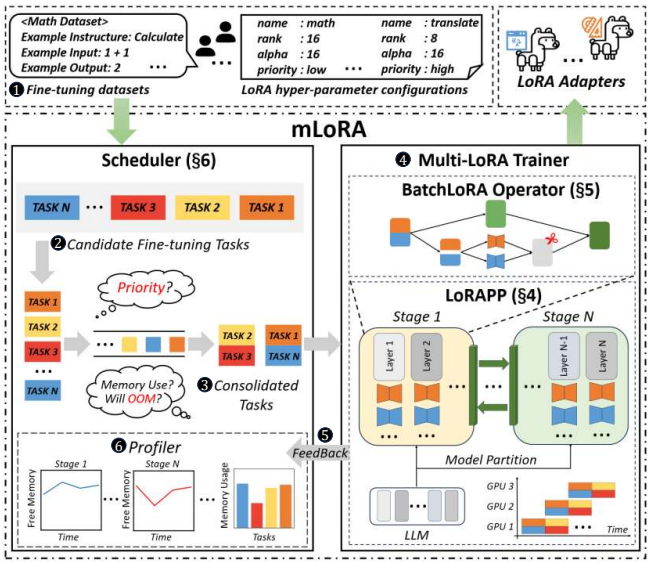

Figure 1: Overview of mLoRA.

three) micro-batches to reduce the bubbles. However, despite the achievement of a zero-bubble pipeline without compromising model performance, the GPU utilization remains suboptimal, primarily due to the small size of individual tasks. To enhance utilization, we opt to execute multiple tasks concurrently. However, launching multiple tasks simultaneously will result in frequent GPU kernel launches, which is inefficient. To address this issue, we introduce a novel operator, BatchLoRA. This operator aggregates multiple LoRA tasks and performs batched matrix multiplications, significantly reducing kernel launch overhead and drastically decreasing overall training time. In addition, the independence of training multiple LoRA adapters also enables the opportunity to overlap GPU communication and computation. Such an overlap greatly hides the I/O latency from GPU computation, further improving the utilization of GPUs.

mLoRA effectively addresses the challenges of limited access to high-end GPUs and the high computational cost of large-cale fine-tuning. It has been deployed in the production environment at AntGroup, reducing the time required for hyperparameter search in LLM fine-tuning by 30%. This demonstration will: (1) introduce the technical details of the two core components in mLoRA, (2) explain how users can efficiently fine-tune their LoRA tasks in parallel using mLoRA, and (3) present the main uses and benefits of mLoRA in the database domain (LLM4DP [6] and Text2SQL [3]).

2 SYSTEM OVERVIEW

Architecture Overview. As shown in *Figure* 1, mLoRA comprises two main components: 1) a multi-LoRA trainer designed for parallel operation, which handles multiple LoRA fine-tuning tasks concurrently and performs runtime optimization through BatchLoRA and LoRAPP technologies, and 2) a task scheduler that selects a batch of LoRA tasks based on user requirements and metrics from the performance profiler, such as scheduling tasks to maximize GPU resource utilization and reduce out-of-memory (OOM) occurrences.

Workflow. *Figure* 1 presents the mLoRA workflow. ❶ Users send requests to mLoRA by providing hyperparameter configurations for LoRA adapters along with the datasets used for fine-tuning. ❷ mLoRA then generates candidate tasks with their initial configurations and places them in candidate task queue. ❸ The task scheduler selects a batch of tasks for parallel training from the candidate task queue, taking into account factors such as memory usage and task priority. ❹ The tasks are scheduled concurrently across different stages of fine-tuning using a pipeline parallelism mechanism (LoRAPP) that achieves a zero-bubble schedule. In addition, mLoRA uses the BatchLoRA operator to aggregate tasks into a large batch and performs collective matrix multiplication operations for all adapters. This approach reduces kernel launch overhead and improves GPU efficiency while preserving model quality. ❺ During training, the multi-LoRA trainer reports performance metrics, including the actual memory usage of each task, to the profiler. ❻ The profiler uses this information to continually update its memory estimation model, allowing for more precise evaluations of memory requirements for future tasks.

Components. In the following sections, we discuss the three core components—LoRAPP, BatchLoRA, and the Task Scheduler—that enable mLoRA to perform efficient parallel training of LoRA tasks. For additional technical details and experimental results, please refer to our research paper.

3 DESIGN OF MLORA

3.1 LoRAPP: Multi-LoRA Training Parallelism

When fine-tuning multiple LoRA tasks exceeds the capacity of a single GPU, parallelization across multiple GPUs becomes necessary. Common approaches include data parallelism (DP), tensor parallelism (TP), and pipeline parallelism (PP). However, storing the full model on each GPU in DP makes the training and fine-tuning of large models inefficient and sometimes infeasible. TP, on the other hand, incurs high communication overhead. While PP reduces communication costs, it still encounters idle periods due to computational dependencies. Variants such as GPipe and PipeDream attempt to mitigate pipeline stalls by using micro-batches or relaxing dependency constraints, but these methods neither fully eliminate stalls nor preserve model performance.

In contrast, mLoRA introduces a novel pipeline mechanism, LoRAPP, which leverages the independence between multiple adapters to achieve a stall-free pipeline without degrading the model's convergence. LoRAPP fills the pipeline by executing different stages from independent adapters, thus maintaining a stall-free pipeline when task load is sufficient. This approach effectively preserves the model's performance. When the LoRA tasks are insufficient to fully occupy the pipeline, LoRAPP uses the GPipe method to split a single batch into smaller micro-batches, reducing idle times. Additionally, it prioritizes interleaving the backward phases to release memory earlier, enabling the support for more fine-tuning tasks.

The independence of training multiple LoRA adapters also enables overlapping GPU communication and computation. For example, since there is no dependency between the i-th and j-th LoRA adapters, while the backward pass of the ith adapter is executed on GPU $K+1$, the forward pass of the jth adapter can be concurrently processed on GPU K. This approach effectively hides the

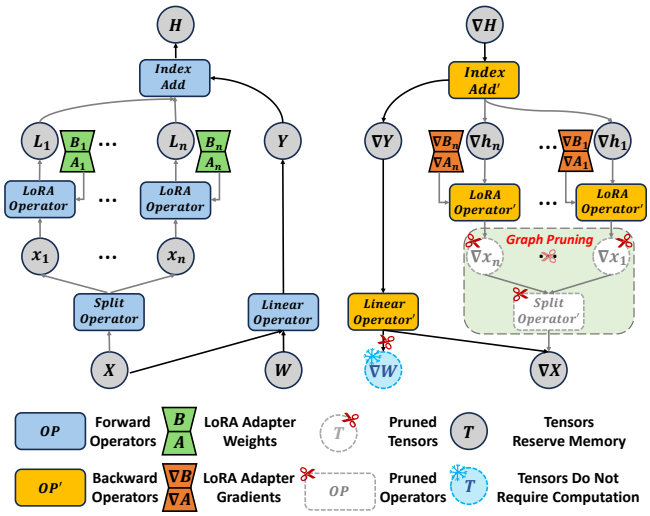

Figure 2: Computational graphs of BatchLoRA operator with graph pruning.

communication overhead and maximizes overall GPU utilization efficiency.

3.2 BatchLoRA: Multi-LoRA Training Operator

After eliminating bubbles and hiding the I/O communication delays, LoRAPP enables efficient pipeline fine-tuning across GPUs. However, unlike complex tasks such as full-scale LLM training, each LoRA fine-tuning task is relatively simple, with a smaller memory footprint. A LoRA task in each pipeline step monopolizes the GPUs. However, it cannot fully utilize their memory and computational resources. To further improve GPU efficiency and utilization, the most straightforward approach is to schedule as many LoRA tasks as possible simultaneously, fully leveraging the GPU's parallel processing capability. However, launching multiple LoRA tasks in a simple loop manner will frequently trigger GPU kernel launches. Although the cost of each launch is small, this overhead can account for over 10% of the total training time. To address this, we designed BatchLoRA, which consolidates multiple LoRA tasks into a large batch and performs collective matrix multiplication operations for all the involved adapters, significantly reducing kernel overhead.

As shown in Figure 2, the left image depicts a user-defined forward propagation graph and the right image illustrates an automatically generated backward propagation graph. BatchLoRA combines multiple LoRA parameter matrices into a single large matrix for computing $(x_1, \ldots, x_n)$. Compared to simply launching multiple LoRA tasks in a loop manner, BatchLoRA can reduce the launch overhead associated with $K - 1$ times of computation related to the base weight, where K is the number of LoRA tasks.

Additionally, the BatchLoRA operator constructs a more efficient computational graph to minimize the unnecessary backward propagation overhead, instead of relying on an automatically generated graph. The right panel of Figure 2 shows the details of our pruning strategy. For further information, please refer to our full research paper.

3.3 Task Scheduler

The goal of mLoRA is to maximize GPU utilization by fine-tuning as many tasks as possible. Thus, accurate task memory prediction and efficient scheduling algorithms are critical. We schedule tasks based on the preemptive priorities of LoRA tasks, applying a first-come, first-served policy when priorities are the same. During the system processing task, mLoRA continuously collects metrics through profiling and uses a non-linear least squares solver to fit a coefficient relationship between the memory footprint and the size of the task's training input data, following the model described by Vijay et al. [4]. Using historical data, mLoRA iteratively refines the coefficient selection for model fitting. This process enables accurate memory usage estimation, maximizing GPU utilization while effectively preventing out-of-memory (OOM) errors.

4 DEMONSTRATION SCENARIOS

This section presents two fine-tuning scenarios to demonstrate how mLoRA enables efficient model adaptation in the database domain. The Text2SQL scenario highlights mLoRA's ability to optimize the search for model parameters for improved performance, while the LLM4DP scenario demonstrates how mLoRA effectively builds workflows for Database Processing (DP) tasks.

Scenario 1: Text2SQL Fine-tuning LLMs with LoRA has become a new paradigm for solving Text2SQL tasks [3]. However, achieving optimal model performance often incurs significant time and financial costs due to the large number of parameters involved in the fine-tuning process, such as rank, alpha, and learning rate ratio. mLoRA allows users to concurrently train multiple LoRA tasks, accelerating the search for the best-performing model.

For example, as shown in Figure 3, the user provides a subset of the BIRD [5] dataset in A and submits a description of the data format in the prompt. In B, the user builds the BIRD dataset and sets up the data preprocessing method. Next, the user configures the adapters required for LoRA training in C and binds the adapters to the specified training task in D. During this process, the user sets more than ten training hyperparameters, sets over ten training hyperparameters, including learning rate, optimizer, and batch size. mLoRA enables users to submit multiple training tasks simultaneously and run them in parallel. As shown in E, the user submits six tasks. During parallel training, the system provides real-time metrics, including GPU utilization, memory usage, and the loss curve for each task. Additionally, mLoRA provides a one-click deployment feature in F, allowing users to quickly verify and identity the optimal parameter configuration.

By leveraging mLoRA, the time required to identify the optimal parameters for the Text-to-SQL task is reduced by 30%. Furthermore, when training more than six tasks, the overall training time decreases by an additional 10%.

Scenario 2: LLM4DP With reference to the scenario described in Jellyfish [6], we have constructed a universal data processing worker that includes four downstream tasks with their datasets provided in Jellyfish[2]: Error Detection (ED), Data Imputation (DI), Schema Matching (SM), and Entity Matching (EM). These tasks share the same base model but are built with different datasets and configurations. Therefore, we uploaded these four datasets

[2]https://huggingface.co/datasets/NECOUDBFM/Jellyfish-Instruct/tree/main

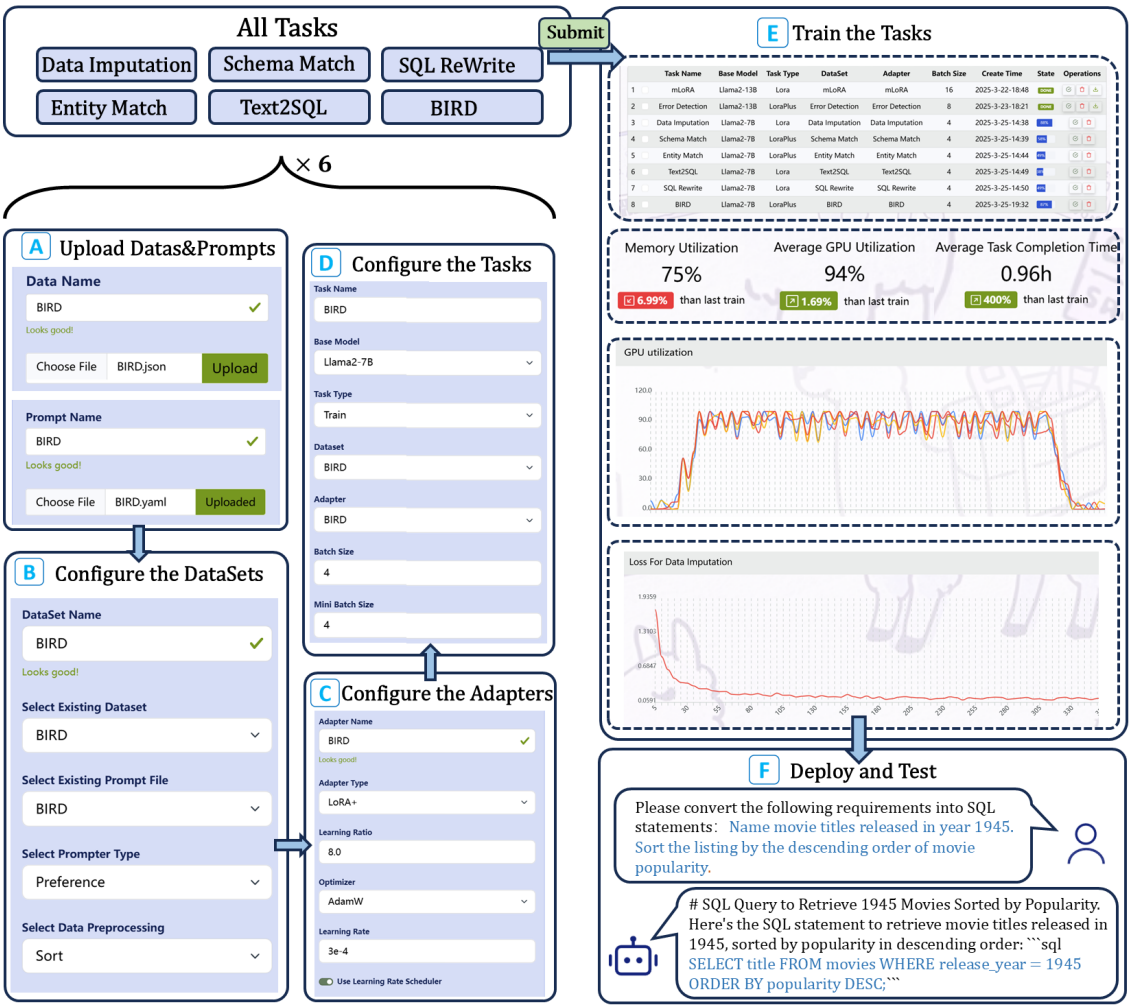

Figure 3: mLoRA Demonstration

and prompt files simultaneously in A, built the four datasets in B, created four adapters for these tasks in C and submitted all four tasks for training simultaneously after their creations. During the parallel training of these tasks, we observed high GPU utilization in D, and when we encountered an erroneous loss curve in E, we can resubmitted the task. The newly submitted task waited for scheduling and trained in parallel. After training was completed, we deployed the adapter and base model in F and conducted brief tests on them with a simple data processing task to verify the training success. The parallel training method provided by mLoRA improves the training speed by a factor of 4 to 5 compared to serial training.

These tasks share the same base model but are constructed with different datasets and configurations, making them ideal for turning on training for these tasks in parallel. Using mLoRA, training is 30%~45% faster compared to methods like 1F1B or TP, making it possible to quickly fine-tuning multiple downstream tasks in data preprocessing with minimal effort.

REFERENCES

[1] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *Proceedings of the VLDB Endowment* 17, 2 (2023), 92–105.

[2] Benjamin Feuer, Yurong Liu, Chinmay Hegde, and Juliana Freire. 2024. ArcheType: A Novel Framework for Open-Source Column Type Annotation using Large Language Models. *Proc. VLDB Endow.* 17, 9 (2024), 2279–2292. https://www.vldb.org/pvldb/vol17/p2279-freire.pdf

[3] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1132–1145.

[4] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023).

[5] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).

[6] Haochen Zhang, Yuyang Dong, Chuan Xiao, and Masafumi Oyamada. 2023. Jellyfish: A Large Language Model for Data Preprocessing. *arXiv preprint arXiv:2312.01678* (2023).